

Quickstart — adar-shape-pipeline

This walkthrough runs the whole pipeline on the bundled **STK4 AluSq2** demo data and reproduces the skyline figure at the end. It takes ~15–20 minutes, most of it ShapeMapper.

The pipeline measures the RNA structure of the **ADAR-edited** subpopulation of molecules: it runs ShapeMapper, sorts reads by A→G editing at a cutoff you choose, reverts the edits, re-probes, and normalizes — so the edited-molecule reactivity is directly comparable to the bulk SHAPE profile.

1. Set up

```
git clone https://github.com/alaederach/adar-shape-pipeline.git
cd adar-shape-pipeline

# Python/CLI dependencies
conda env create -f environment.yml
conda activate shape-adar

# ShapeMapper 2.2 (not on bioconda; x86-64 Linux only)
#   native Linux : install it, then set --shapemapper_bin (or env ADAR_SHAPEMAPPER_BIN)
#   macOS/other  : build the bundled container once and use -profile docker
docker build --platform linux/amd64 -t shape-adar/shapemapper:2.2.0 .

# Demo FASTQs (hosted on UNC Dataverse, doi:10.15139/S3/E2BTCM)
bin/download_data.sh
```

The demo FASTQs are **downsampled (~300k read pairs/file)** so this runs fast. The full-depth data goes to GEO/SRA on publication.

2. Step 01 — ShapeMapper profiles

```
nextflow run main.nf -profile docker \
  --samplesheet input/samplesheet.csv --outdir results
```

This produces, under `results/`:

- `shapemapper/<sample>/..._profile.txt` — a reactivity profile per sample (ADAR-SHAPE, ADAR-DMSO, standard-SHAPE)
- `edit_positions/<sample>_ranked_edit_positions.tsv` — A→G positions ranked by editing rate

(On a native-Linux host use `-profile standard`; on a SLURM cluster use `-profile slurm`.)

3. Choose a sorting cutoff

Open the ADAR-DMSO ranked table — it lists each A position and its editing rate:

```
column -t results/edit_positions/STK4_AluSq2_ADAR-DMSO_rep1_ranked_edit_positions.tsv | head
```

For this construct the strong sites (105, 115, 118) edit at ~16–59%, well above the noise. The demo uses a cutoff of `min_rate = 0.15` (already set in `input/samplesheet_step02.csv`), which keeps those high-confidence positions. You could instead list explicit positions in the `candidates` column (see `input/samplesheet_step02_candidates.csv`).

4. Step 02 — sort, deconvolve, normalize, plot

```
nextflow run step02.nf -profile docker \  
  --samplesheet input/samplesheet_step02.csv \  
  --step01_outdir results --outdir results
```

Outputs under `results/reactivity_profiles/STK4_AluSq2/minrate15/`:

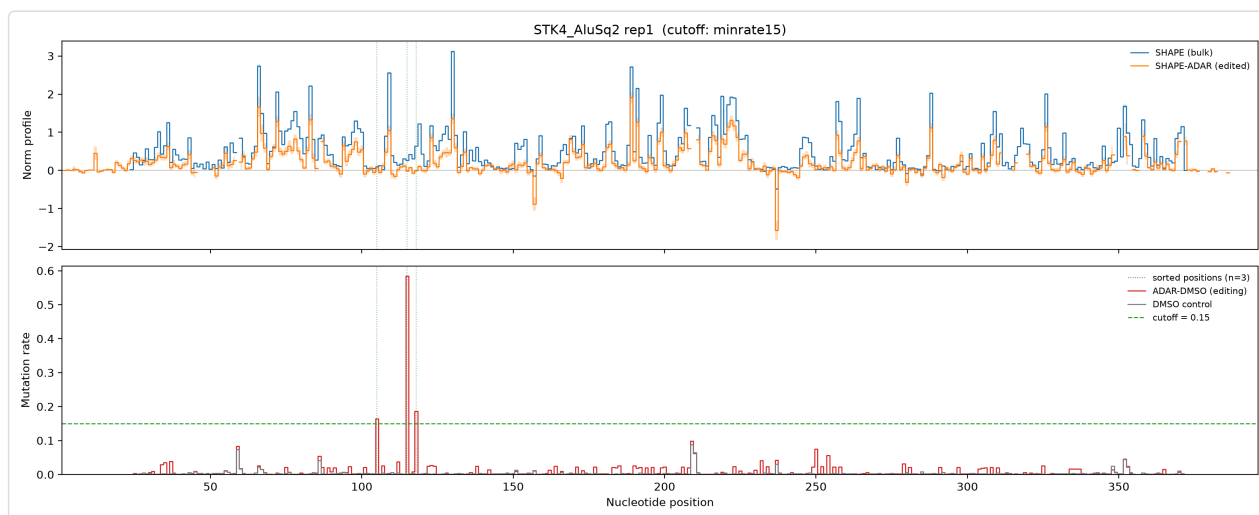
- `..._edited_normalized_profile.txt` — **the headline result**: deconvolved, normalized reactivity of the edited molecules (use the `Norm_profile` column)
- `..._reference_normalized_profile.txt` — the bulk reference on the same scale
- `..._edited_skyline.png` — the control plot below

5. Read the result

Your run writes the skyline to

`results/reactivity_profiles/STK4_AluSq2/minrate15/STK4_AluSq2_rep1_minrate15_edited_skyline.png`.

Open it — the two panels show:



Example: a successful demo run (downsampled data, cutoff 0.15).

Top panel — reactivity: bulk SHAPE (blue) vs the deconvolved SHAPE-ADAR edited-molecule reactivity (orange). They track together (same overall fold) and diverge at the positions where editing remodels the structure.

Bottom panel — the basis for sorting: ADAR-DMSO editing rate (red) vs the non-ADAR DMSO control (grey). The horizontal line is the cutoff; the vertical lines (through both panels) mark the positions reads were sorted on.

6. Did it work?

A successful demo run gives:

- `nextflow run ...` ends with no `ERROR` and all processes `Completed`
- `results/reactivity_profiles/STK4_AluSq2/minrate15/` contains the two profile `.txt` files and the `..._skyline.png`
- the edited `Norm_profile` column has real (non- `nan`) values across most of the construct
- the skyline matches the figure above

If the normalized profile is mostly `nan` , the edited read set was too shallow — lower the cutoff (`min_rate`) or use deeper data. The sorter also prints a warning in that case.